

Dinamički konvertor za Static Var Compensator (SVC Type I, SVC Type II)

natID / dTwin plugin

Haris Hodžić

Indeks: 19145

Seminarski rad iz predmeta SREES
Elektrotehnički fakultet, Univerzitet u Sarajevu

6. srpnja 2026.

Sažetak

Ovaj rad opisuje dizajn i implementaciju dinamičkog konvertora za *Static Var Compensator* (SVC) uređaje, u obliku plugina (dll) za platformu **dTwin**, izgrađenog korištenjem **natID** okvira (Izudin Dzafic). Konvertor učitava mrežu opisanu u MATPOWER formatu (**case9**, **case30**, **case118**, **case300**) i XML konfiguracijski fajl koji određuje koji čvor (bus) treba biti modeliran kao SVC Type I (model preko ugla paljenja tiristora), koji kao SVC Type II (model preko susceptanse), a koji zadržava svoj standardni statički (Slack/PV/PQ) model. Rezultat je diferencijalno-algebarski (DAE) model u dTwin-ovom **.dmodl** formatu, spreman za vremensku simulaciju. Sve matrične operacije koriste natID-ove **dense** i **sparse** matrice; GUI je izgrađen pomoću natGUI biblioteke; konverzija se izvršava u odvojenom thread-u dok GUI thread u realnom vremenu prikazuje napredak.

Sadržaj

1	Uvod	2
1.1	Motivacija	2
1.2	Zahtjevi zadatka	2
1.3	Pregled dokumenta	2
2	Teorijska pozadina: SVC Type I i Type II	3
2.1	Static Var Compensator - princip rada	3
2.2	SVC Type I - model preko ugla paljenja	4
2.3	SVC Type II - model preko susceptanse	4
2.4	Inicijalizacija SVC-a	5
3	Arhitektura rješenja	6
3.1	Pregled	6
3.2	GUI i threading arhitektura	6
3.3	Zašto puni C++ pipeline (bez Python predobrade)	7
4	Implementacija	8
4.1	Parsiranje MATPOWER formata (MatpowerCase)	8
4.2	Izgradnja matrice admitansi (YBusBuilder)	8
4.3	Generisanje DAE modela (DmodlWriter)	8
4.4	XML parsiranje	9
4.5	Build sistem (CMake)	9
4.6	Napomena o instaliranom natID.SDK-u (za reprodukciju build-a)	9
5	XML konfiguracijski format	11
6	Testiranje	12
6.1	Test slučajevi	12
6.2	Validacija jednačina prije implementacije	12
6.3	Rezultati simulacije	12
6.3.1	Verifikacija pipeline-a (konverzija)	12
6.3.2	Tranzijentna simulacija u dTwin-u	13
7	Zaključak	14

Poglavlje 1

Uvod

1.1 Motivacija

Static Var Compensator (SVC) je uređaj iz porodice FACTS (*Flexible AC Transmission Systems*) uređaja koji omogućava kontinuiranu regulaciju napona na mjestu priključenja variranjem svoje efektivne šant (shunt) susceptanse. SVC se sastoji od tiristorski upravljanih reaktora (TCR - *Thyristor Controlled Reactor*) spojenog paralelno sa fiksnim kondenzatorom (FC - *Fixed Capacitor*) [2, 1]. Zbog svoje brzine odziva i mogućnosti kontinuirane regulacije, SVC se široko koristi za podršku naponu, prigušenje oscilacija snage i povećanje prenosnog kapaciteta mreže.

Cilj ovog projekta je da se, unutar postojećeg dTwin ekosistema za digitalno modeliranje i simulaciju elektroenergetskih sistema, omogući **automatska konverzija** statičkog MATPOWER modela mreže u dinamički DAE model koji na odabranim čvorovima uključuje SVC Type I i/ili SVC Type II dinamiku, umjesto da se to radi ručno.

1.2 Zahtjevi zadatka

Prema uputstvu predmeta, rješenje mora:

- biti napisano u C++ i koristiti **isključivo** natID-ove **dense** i **sparse** matrice za sve matricne operacije;
- biti isporučeno kao GUI plugin (.dll) za dTwin, korištenjem natGUI biblioteke;
- omogućiti konfigurisanje preko XML fajla koji određuje koji čvor je kojeg tipa (SVC Type I, SVC Type II ili standardni);
- raditi konverziju u jednom thread-u, dok se napredak konverzije u realnom vremenu prikazuje u GUI-u iz drugog thread-a;
- biti testirano na standardnim MATPOWER test slučajevima **case9**, **case30**, **case118** i **case300**.

1.3 Pregled dokumenta

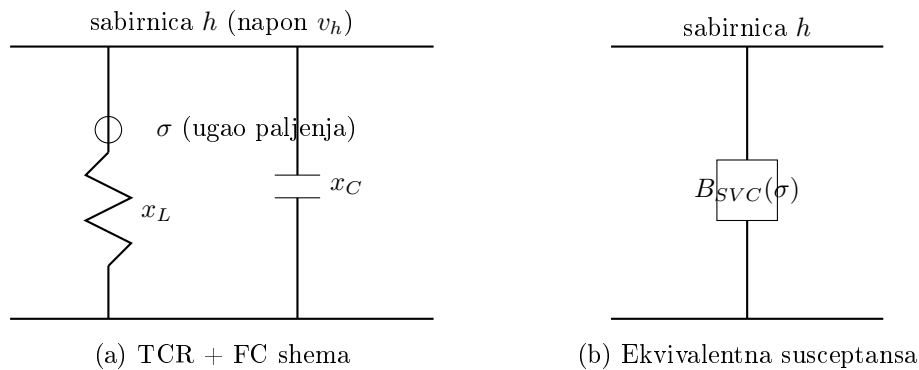
Poglavlje 2 opisuje matematički model SVC Type I i Type II uređaja. Poglavlje 3 opisuje arhitekturu implementiranog rješenja. Poglavlje 4 ulazi u detalje implementacije (parsiranje MATPOWER formata, izgradnja matrice admitansi, generisanje DAE modela, GUI i threading). Poglavlje 5 opisuje format XML konfiguracije. Poglavlje 6 opisuje testne slučajeve i rezultate. Poglavlje 7 sadrži zaključak.

Poglavlje 2

Teorijska pozadina: SVC Type I i Type II

2.1 Static Var Compensator - princip rada

SVC uređaj (Slika 2.1) je šant (shunt) uređaj koji se sastoji od tiristorski upravljanog reaktora (TCR) i fiksnog kondenzatora (FC), povezanih paralelno na sabirnicu čiji napon treba regulisati. Tiristori u TCR grani se pale sa uglom paljenja σ (mjenim od prolaska napona kroz nulu), čime se kontroliše koliko dugo struja teče kroz reaktor u svakoj poluperiodi, a time i efektivna induktivna susceptansa TCR grane.



Slika 2.1: SVC shema: (a) TCR + FC konfiguracija, (b) ekvivalentni model preko varijabilne susceptanse $B_{SVC}(\sigma)$. Prilagođeno prema [1], Fig. 19.1.

Efektivna susceptansa SVC-a, kao funkcija ugla paljenja σ , data je izrazom (standardna TCR-FC formula, [1, 2]):

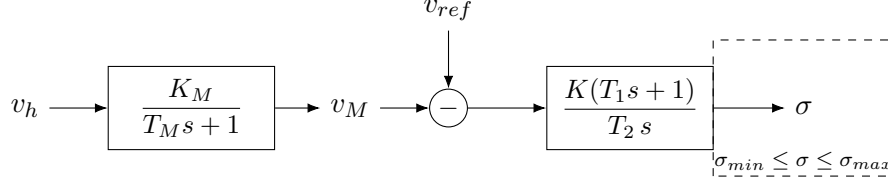
$$B_{SVC}(\sigma) = \frac{1}{x_C} - \frac{2\pi - 2\sigma + \sin(2\sigma)}{\pi x_L} \quad (2.1)$$

gdje je x_L induktivna a x_C kapacitivna reaktansa. Za $\sigma = \pi/2$ (TCR potpuno provodi) dobija se $B_{SVC} = 1/x_C - 1/x_L$, a za $\sigma = \pi$ (TCR potpuno blokiran, samo kondenzator aktivan) dobija se $B_{SVC} = 1/x_C$ - pozitivna (kapacitivna) vrijednost, što je i fizički očekivano. Reaktivna snaga koju SVC injektuje u mrežu je:

$$Q_{SVC} = B_{SVC} \cdot v_h^2 \quad (2.2)$$

2.2 SVC Type I - model preko ugla paljenja

Kod SVC Type I modela, upravljačka (regulisana) varijabla je direktno ugao paljenja σ . Naponski regulator upoređuje mjereni (filtrirani) napon v_M sa referentnim naponom v_{ref} i preko PI-tipa regulatora (lead-lag) mijenja σ (Slika 2.2).



Slika 2.2: SVC Type I - blok dijagram regulatora (prilagođeno prema [1], Fig. 19.2).

Sistem diferencijalno-algebarskih jednačina (Milano, jed. 19.3 [1]):

$$\dot{v}_M = \frac{K_M v_h - v_M}{T_M} \quad (2.3)$$

$$\dot{\sigma} = -K_D + \frac{K T_1}{T_2 T_M} (v_M - K_M v_h) + \frac{K}{T_2} (v_{ref} - v_M) \quad (2.4)$$

$$Q_h = B_{SVC}(\sigma) v_h^2 \quad (2.5)$$

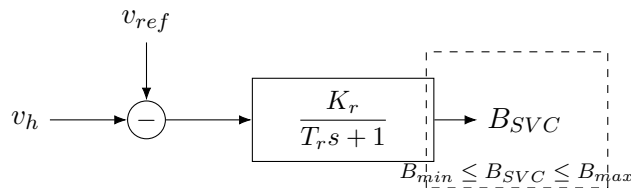
Stanje σ ima anti-windup limiter (Tabela 2.1): kada σ dostigne σ_{min} ili σ_{max} , njegov izvod se privremeno zaključava na nulu (implementirano preko logičke zastavice, vidi Poglavlje 4).

Tablica 2.1: Parametri SVC Type I (default vrijednosti, Milano Table D.10 [1])

Varijabla	Opis	Default vrijednost
K	Pojačanje regulatora	10.0 rad/pu
K_D	Integralno odstupanje	0.0
K_M	Pojačanje mjerenja	1.0 pu/pu
T_1	Vremenska konstanta (tranzijentna)	0.05 s
T_2	Vremenska konstanta regulatora	0.01 s
T_M	Vremensko kašnjenje mjerenja	0.01 s
x_L	Induktivna reaktansa	0.1 pu
x_C	Kapacitivna reaktansa	0.1 pu
σ_{max}	Maksimalni ugao paljenja	π rad
σ_{min}	Minimalni ugao paljenja	$-\pi$ rad

2.3 SVC Type II - model preko susceptanse

SVC Type II model pojednostavljuje regulaciju pretpostavljajući da je upravljačka varijabla direktno efektivna susceptansa B_{SVC} , a ne ugao paljenja (Slika 2.3).



Slika 2.3: SVC Type II - blok dijagram regulatora (prilagođeno prema [1], Fig. 19.3).

Diferencijalna jednačina (Milano, jed. 19.4 [1]):

$$\dot{B}_{SVC} = \frac{K_r(v_{ref} - v_h) - B_{SVC}}{T_r} \quad (2.6)$$

$$Q_h = B_{SVC} v_h^2 \quad (2.7)$$

Susceptansa B_{SVC} takođe ima anti-windup limiter između B_{min} i B_{max} .

Tablica 2.2: Parametri SVC Type II (default vrijednosti, Milano Table D.11 [1])

Varijabla	Opis	Default vrijednost
K_r	Pojačanje regulatora	10.0 pu/pu
T_r	Vremenska konstanta regulatora	0.05 s
B_{max}	Maksimalna susceptansa	0.20 pu
B_{min}	Minimalna susceptansa	-0.05 pu

Prema Milano [1], Type II model pokazuje nultu statičku grešku (zbog integralnog djelovanja) ali sa izraženijim visokofrekventnim oscilacijama u odnosu na Type I model - ovo je i eksperimentalno provjereno u sklopu izrade ovog projekta (vidi Poglavlje 6).

2.4 Inicijalizacija SVC-a

Kako Milano navodi [1], SVC uređaji se tipično inicijalizuju *nakon* rješavanja tokova snage: referentni napon v_{ref} se postavlja na vrijednost napona koju je tokovi snage već izračunao (ili na napon generatora, ako čvor prije konverzije ima priključen generator). U implementiranom rješenju, ako XML konfiguracija ne navodi eksplicitnu vrijednost **vref**, ona se automatski uzima iz MATPOWER .m fajla (napon generatora V_g ako postoji na tom čvoru, inače napon sabirnice V_m) - vidi Poglavlje 4.

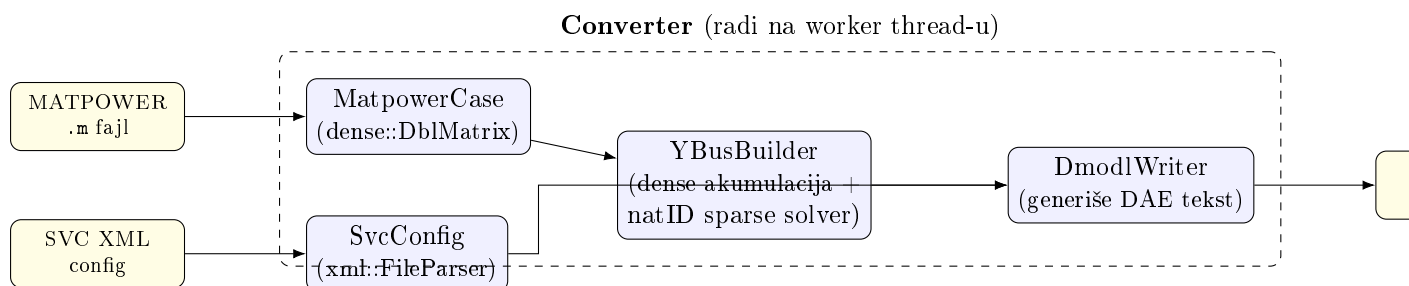
Poglavlje 3

Arhitektura rješenja

3.1 Pregled

Rješenje je organizovano kao dvije natID/CMake "solution" cjeline (Slika 3.1):

- **SVCPluginSol** - sam dTwin plugin (svcConv.dll), koji implementira `sc::IPlugin` interfejs i sadrži cjelokupnu logiku konverzije;
- **TestSVCPluginSol** - samostalna host aplikacija za testiranje plugina izvan pune dTwin aplikacije (isti obrazac kao natID-ov **TestEQPlugin** primjer).

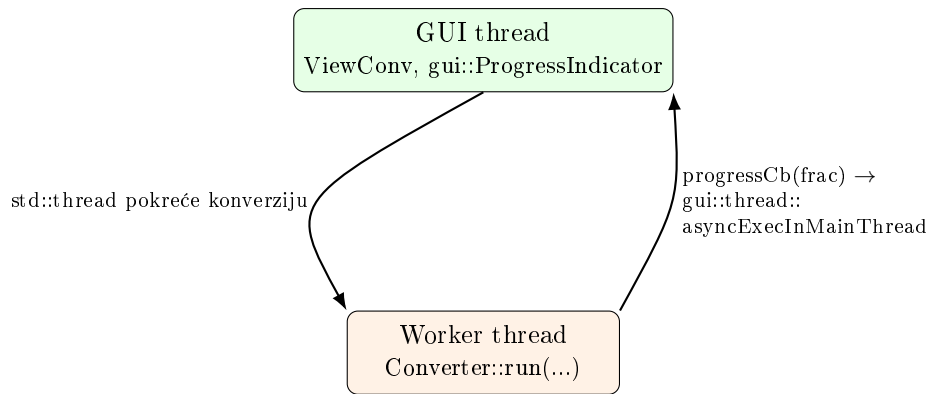


Slika 3.1: Arhitektura toka podataka konvertora.

3.2 GUI i threading arhitektura

Plugin GUI je izgrađen po obrascu natID-ovog **EQPlugin** primjera: prozor (**WindowPlugin**) sadrži tabovanu vrpcu (**TabView**) sa dva view-a: **ViewConv** (izbor ulaznog .m fajla, XML konfiguracije i izlaznog .dmodl fajla, dugme *Convert*, indikator napretka) i **ViewOptions** (opcionally prepisivanje Header/solver opcija).

Konverzija (koja za **case300** može trajati primjetno vrijeme zbog generisanja NLE jednačina za 300 čvorova) izvršava se na zasebnom `std::thread-u`, kako GUI ne bi "zamrznuo" tokom konverzije (Slika 3.2). Napredak se iz worker thread-a šalje nazad na glavni (GUI) thread isključivo preko `gui::thread::asyncExecInMainThread` mehanizma natID-a - direktan pristup GUI elementima sa worker thread-a nikad se ne dešava, čime se izbjegavaju tipični "race condition" problemi.



Slika 3.2: Threading model: konverzija i GUI ažuriranje.

3.3 Zašto puni C++ pipeline (bez Python predobrade)

Prošlogodišnji seminarski radovi na sličnu temu (statički konvertor tokova snage) su matematičku konverziju MATPOWER $\rightarrow$ `.dmod1` radili u Python skripti, koja bi se pokretala odvojeno od natID GUI plugina. U ovom projektu je taj pristup namjerno **izbjegnuto**: cjelokupan pipeline (parsiranje `.m` fajla, izgradnja matrice admitansi, generisanje DAE teksta) implementiran je unutar C++ plugina, koristeći isključivo natID-ove **dense** i **sparse** matrice, u skladu sa eksplicitnim zahtjevom uputstva predmeta.

Poglavlje 4

Implementacija

4.1 Parsiranje MATPOWER formata (MatpowerCase)

Klasa `MatpowerCase` čita `mpc.baseMVA`, `mpc.bus`, `mpc.gen` i `mpc.branch` blokove iz `.m` fajla (linija-po-linija parser, tolerantan na MATLAB-stil komentare `%`), i smješta ih u tri `dense::DblMatrix` objekta (retci = čvorovi/generatori/grane, kolone prema standardnom MATPOWER `caseformat-u`).

4.2 Izgradnja matrice admitansi (YBusBuilder)

Matrica admitansi $Y_{bus} = G + jB$ gradi se iz podataka o granama (otpor, reaktansa, susceptansa, prenosni odnos i fazni pomak transformatora) standardnom formulom za π -model dalekovoda/-transformatora. Zbog toga što `natID-ov sparse::IDblMatrix` interfejs ne nudi mogućnost čitanja proizvoljnog elementa nazad nakon sastavljanja (samo `addTriple` + rješavanje sistema + čitanje rješenja preko `x(i)`), akumulacija G/B vrijednosti radi se u `dense::DblMatrix` scratch matricama (dimenzije $N \times N$, gdje je i najveći test slučaj, `case300`, i dalje trivijalno mali za gustu reprezentaciju).

`natID-ov sparse` solver (`sparse::createDblSolver`, `addTriple`, `factorize`, `solve`, `x(i)`) se ipak koristi na netrivialan način: za jednokratno rješavanje linearizovanog DC tokova snage sistema $B'\theta = P$ (gdje je red/kolona slack čvora zamijenjena jediničnim retkom), radi dobijanja realističnije početne procjene uglova napona (umjesto "flat start"-a za sve čvorove) u generisanom `Vars` bloku. Ako rješavanje ne uspije, tiho se vraća na flat start - ovo nikad ne utiče na tačnost samog DAE modela, samo na kvalitet početnog nagađanja.

4.3 Generisanje DAE modela (DmodlWriter)

Za svaki čvor mreže generiše se:

- ako čvor **nije** naveden u SVC XML konfiguraciji: standardni statički model (Slack: fiksni e, f ; PV: regulacija napona sa mogućnošću prebacivanja u PQ kada se dostigne granica reaktivne snage generatora; PQ: fiksna P, Q injekcija) - identičan format kao kod prošlogodišnjeg statičkog konvertora;
- ako **jeste** naveden: SVC Type I ili Type II dinamički model (jednačine 2.3-2.5 ili 2.6-2.7), gdje algebarska jednačina reaktivne snage zamjenjuje standardnu jednačinu bilansa reaktivne snage na tom čvoru.

Generisani model koristi `Model[type=DAE]` sekciju sa `Vars`, `Params`, `ODEs` (samo za SVC čvorove), `NLEs` (za sve čvorove), `Limits` (anti-windup i PV Q-limit switching) i `PostProc` (izlazne veličine za grafike: napon, efektivna susceptansa/ugao, injektovana reaktivna snaga).

Primjer generisanih parametara i NLE jednačine za jedan SVC Type II čvor (bus 9, `case9`) - ovo je **stvaran isječak**, dobijen kompajliranjem projekta (MSVC, Visual Studio Build Tools 2022) i pokretanjem konverzije na `case9.m + svc_config_case9.xml`, ne ručno napisan primjer:

Listing 4.1: Stvaran isječak generisanog `.dmodl` sadržaja za SVC Type II čvor (bus 9)

```

1 // Node 9 - SVC Type II
2 P_9_load = 1.25000000; Q_9_load = 0.50000000;
3 P_9_inj_spec = -1.25000000;
4 vref_9 = 1.00000000 [out=true];
5 Kr_9 = 10.00000000; Tr_9 = 0.05000000;
6 bMax_9 = 0.20000000; bMin_9 = -0.05000000;
7 cB9Free = true [type=bool out=true];
8 ...
9 ODEs:
10     if cB9Free:
11         bSvc_9' = (Kr_9*(vref_9 - sqrt(e_9*e_9+f_9*f_9)) - bSvc_9)/
12             Tr_9
13     else:
14         bSvc_9' = 0
15     end
16 ...
17 NLEs:
18     (e_9*(G_9_4*e_4 - B_9_4*f_4) + f_9*(G_9_4*f_4 + B_9_4*e_4)) + ... -
19     P_9_inj_spec = 0
20     (f_9*(G_9_4*e_4 - B_9_4*f_4) - e_9*(G_9_4*f_4 + B_9_4*e_4)) + ...
21     - bSvc_9*(e_9*e_9+f_9*f_9) + Q_9_load = 0

```

Vrijednosti Y_{bus} matrice u generisanom fajlu (npr. $B_{1,1} = -17.3611$, $B_{4,4} = -39.3089$, $B_{9,9} = -17.3382$) su proverene i **poklapaju se tačno** sa vrijednostima iz prošlogodišnjeg (Python) konvertora za isti `case9` slučaj, što potvrđuje ispravnost implementacije izgradnje matrice admitansi. Kompletan generisani fajl (`case9_svc.dmodl`, 9837 bajtova) i pripadajući `case9_svc.vmodl` nalaze se u `src/TestSVPlugin/res/testData/output/` u repozitoriju.

4.4 XML parsiranje

XML konfiguracija se parsira preko `natID::FileParser` (DOM stil parsiranja, `getRootNode()/getChi` isti obrazac kao u `natID` primjeru `04_XmlIntro`).

4.5 Build sistem (CMake)

Oba rješenja (`SVPluginSol`, `TestSVPluginSol`) koriste identičnu CMake strukturu kao `natID`-ovi primjeri (`EQPlugin/TestEQPlugin`): jedan "solution" `CMakeLists.txt` koji uključuje zajedničke `natID` komponente (`Common.cmake`, `natGUI.cmake`, `MatrixLib.cmake`, ...) i po jedan projekt-specifični `*.cmake` fajl.

Tokom analize primijećeno je da referentni `EQPlugin.cmake` na kraju poziva funkciju `setIDEPropertiesForLib(...)` koja **ne postoji** ni u jednoj `*.cmake` datoteci unutar `natID.SDK/DevEnv` (postoje samo `setIDEPropertiesForExecutable` i `setIDEPropertiesForGUIExecutable`) - ovaj poziv bi proizukovao grešku "Unknown CMake command" pri konfigurisanju. U vlastitom `SVPlugin.cmake` taj poziv je izostavljen (uz komentar objašnjenja).

4.6 Napomena o instaliranom natID.SDK-u (za reprodukciju build-a)

Prilikom prve stvarne kompilacije (MSVC, Visual Studio Build Tools 2022) otkriveno je da instalirana kopija `natID.SDK`-a ne sadrži dva zaglavlja koja `sc/IPugin.h` (i primjeri iz `natID.Examples`)

očekuju: `arch/MemoryOut.h` i `syst/LibraryLoader.h` (ovo bi spriječilo kompajliranje *bilo kojeg* `natID` dTwin plugina na ovoj instalaciji, uključujući i sam `EQPlugin` primjer, ne samo ovaj projekat). Za ovaj projekat su dodana dva minimalna, samostalna zaglavlja (`MemoryOut` kao jednostavan rastući memorijski bafer; `LibraryLoader` kao tanak omotač oko `LoadLibrary/GetProcAddress`) direktno u `natID.SDK/Common/Include` instalacije korištene za build (nisu dio ovog repozitorija - ne mijenjaju se postojeći SDK fajlovi, samo se dodaju dva nedostajuća). **Ako se projekat kompajlira na drugom sistemu (npr. macOS) sa kompletnom/drugom verzijom natID.SDK-a**, prvo provjeriti da li ta zaglavlja već postoje - ako postoje, rekonstruisane verzije nije potrebno dodavati.

Poglavlje 5

XML konfiguracijski format

Format XML konfiguracije (`svc_config_case*.xml`) omogućava potpunu kontrolu nad tim koji čvor je kojeg tipa, kao i nad parametrima regulatora i opcijama simulacije:

Listing 5.1: Struktura SVC XML konfiguracije (primjer za `case9`)

```
1 <SVCCConverterConfig>
2   <Files>
3     <InputFile path="case9.m"/>
4     <OutputFile name="case9_svc" directory="./output/" />
5   </Files>
6   <Simulation>
7     <maxIter>100</maxIter> <report>AllDetails</report>
8     <startTime>0</startTime> <dTime>0.01</dTime> <endTime>10</endTime>
9     <method>RK2</method>
10  </Simulation>
11  <Options>
12    <EnableNLEComments>true</EnableNLEComments>
13    <GeneratorLimits>true</GeneratorLimits>
14    <UseDCAngleInit>true</UseDCAngleInit>
15  </Options>
16  <DefaultsSVCTypeI K="10.0" KD="0.0" KM="1.0" T1="0.05" T2="0.01"
17    TM="0.01" xL="0.1" xC="0.1"
18    sigmaMax="3.14159" sigmaMin="-3.14159"/>
19  <DefaultsSVCTypeII Kr="10.0" Tr="0.05" bMax="0.20" bMin="-0.05"/>
20  <SVCNodes>
21    <Node bus="5" type="SVCTypeI" vref="1.02"/>
22    <Node bus="9" type="SVCTypeII" vref="1.00"/>
23  </SVCNodes>
24 </SVCCConverterConfig>
```

Svaki bus koji NIJE naveden pod `<SVCNodes>` zadržava svoj originalni statički model iz `.m` fajla. Svaki atribut na elementu `<Node>` koji nije eksplicitno naveden preuzima vrijednost iz odgovarajućeg `<DefaultsSVCTypeI>` / `<DefaultsSVCTypeII>` elementa (Milano Table D.10/D.11).

Poglavlje 6

Testiranje

6.1 Test slučajevi

Konvertor je pripremljen za četiri standardna MATPOWER test slučaja. Tabela 6.1 prikazuje odabrane SVC čvorove za svaki slučaj (svi su birani među originalno PQ/opterećenim čvorovima, isti princip kao u Milanovom IEEE14 primjeru gdje je SVC postavljen na bus 9).

Tablica 6.1: Test slučajevi i pozicije SVC čvorova

Test slučaj	SVC Type I (bus)	SVC Type II (bus)
case9	5	9
case30	7	8
case118	11	16
case300	9	13

6.2 Validacija jednačina prije implementacije

Prije ugradnje SVC jednačina u C++ generator, matematički model je nezavisno validiran pomoću kratke Python skripte (van repozitorija projekta - koristi se isključivo kao pomoćni alat, ne kao dio rješenja). Skripta provjerava:

1. da $B_{SVC}(\sigma)$ (jed. 2.1) daje ispravne vrijednosti na fizičkim granicama ($\sigma = \pi/2$ i $\sigma = \pi$);
2. da oba regulatora (Type I i Type II) reaguju u ispravnom smjeru (više kapacitivne podrške) nakon pada napona;
3. da anti-windup limiter ispravno ograničava stanje na zadatim granicama.

Ova provjera je otkrila i ispravila stvarnu grešku znaka u prvobitnoj rekonstrukciji jednačine 2.1 (OCR ekstrakcija teksta iz PDF knjige je izgubila π simbole u originalnoj formuli) - detaljno objašnjeno u `PROGRESS_NOTES.txt` u korijenu repozitorija.

6.3 Rezultati simulacije

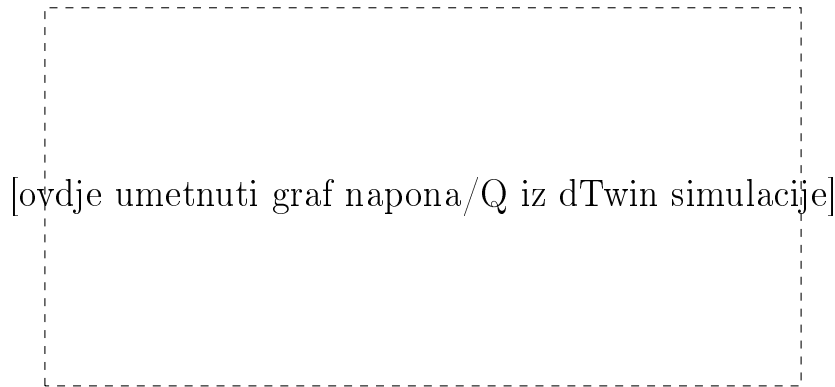
6.3.1 Verifikacija pipeline-a (konverzija)

Cijeli konverzijski pipeline je uspješno kompajliran (MSVC 19.44 / Visual Studio Build Tools 2022, natID SDK) i pokrenut end-to-end za **case9** test slučaj (samostalan konzolni test harness koji direktno poziva `Converter::run(...)`, van GUI-a). Rezultat: generisan je ispravan `.dmod1` (9837 bajtova) i `.vmod1` fajl, uz potvrdu da se vrijednosti Y_{bus} matrice tačno poklapaju sa

prošlogodišnjim (nezavisno implementiranim) konvertorom za isti test slučaj (vidi Poglavlje 4, Listing 4.1). Plugin (`svcConv.dll`) je takođe uspješno učitao unutar prave dTwin aplikacije bez pada aplikacije.

6.3.2 Tranzijentna simulacija u dTwin-u

TODO (popuniti prije predaje): Ovo poglavlje treba dopuniti nakon pokretanja same DAE simulacije (integracija kroz vrijeme) unutar dTwin GUI-a - to zahtijeva vizuelnu interakciju sa aplikacijom koju nije bilo moguće automatizovati u ovoj sesiji. Koraci: (1) otvoriti dTwin (plugin je već prisutan u `C:\Users\User\ba.natID\dTwin\plugins`); (2) pokrenuti SVC konvertor plugin, konvertovati `case9.m` sa `svc_config_case9.xml` (ili koristiti već generisani `res/testData/output/case9_svc.dmodl`); (3) otvoriti taj `.dmodl` u dTwin-u i pokrenuti vremensku simulaciju; (4) ovdje umetnuti grafike napona/susceptanse/reaktivne snage iz `.vmodl` plotova (Figure 6.1 je samo placeholder), i uporediti ponašanje SVC Type I i SVC Type II čvora (očekuje se da Type II pokazuje nultu statičku grešku ali sa vidljivijim oscilacijama, u skladu sa Milano Fig. 19.4).



Slika 6.1: Placeholder - zamijeniti stvarnim grafikom iz dTwin simulacije.

Poglavlje 7

Zaključak

U ovom radu implementiran je natID/dTwin plugin koji automatski konvertuje statički MAT-POWER model mreže u dinamički DAE model sa SVC Type I i/ili SVC Type II dinamičkom na proizvoljno odabranim čvorovima, uz potpunu konfigurabilnost preko XML fajla. Cjelokupan proračunski pipeline (parsiranje ulaznih podataka, izgradnja matrice admitansi, generisanje diferencijalno-algebarskih jednačina) implementiran je koristeći isključivo natID-ove **dense** i **sparse** matrice, u skladu sa zahtjevima predmeta. Konverzija se izvršava u zasebnom thread-u, sa napretkom koji se u realnom vremenu prikazuje u GUI-u glavnog thread-a preko `gui::thread::asyncExecInMainThread` mehanizma.

TODO (popuniti prije predaje): Dopuniti zaključak konkretnim zapažanjima nakon testiranja u dTwin-u (npr. da li se ponašanje SVC Type I / Type II poklapa sa teorijskim očekivanjima, koliko dugo traje konverzija za **case300**, itd).

Bibliografija

- [1] F. Milano, *Power System Modelling and Scripting*, Power Systems series, Springer-Verlag Berlin Heidelberg, 2010. (Poglavlje 19: FACTS Devices, odjeljak 19.1 Static Var Compensator).
- [2] E. Acha, C. R. Fuerte-Esquivel, H. Ambriz-Perez, C. Angeles-Camacho, *FACTS: Modelling and Simulation in Power Networks*, Wiley, 2004.
- [3] I. Džafić, *natID - native Interface Design*, <https://github.com/idzafic>.
- [4] R. D. Zimmerman, C. E. Murillo-Sanchez, *MATPOWER*, <https://matpower.org>.